# **Searching**

# Searching

Searching refers to the operation of finding an item from a list of items based on some key value.

## Two Searching Methods

(1) Linear Search

(2) Binary Search

## Linear Search

- A linear search is a technique for finding a particular value in a unsorted or sorted list.
- Search each item of list using key value, one at a time, until finding the item from the list.

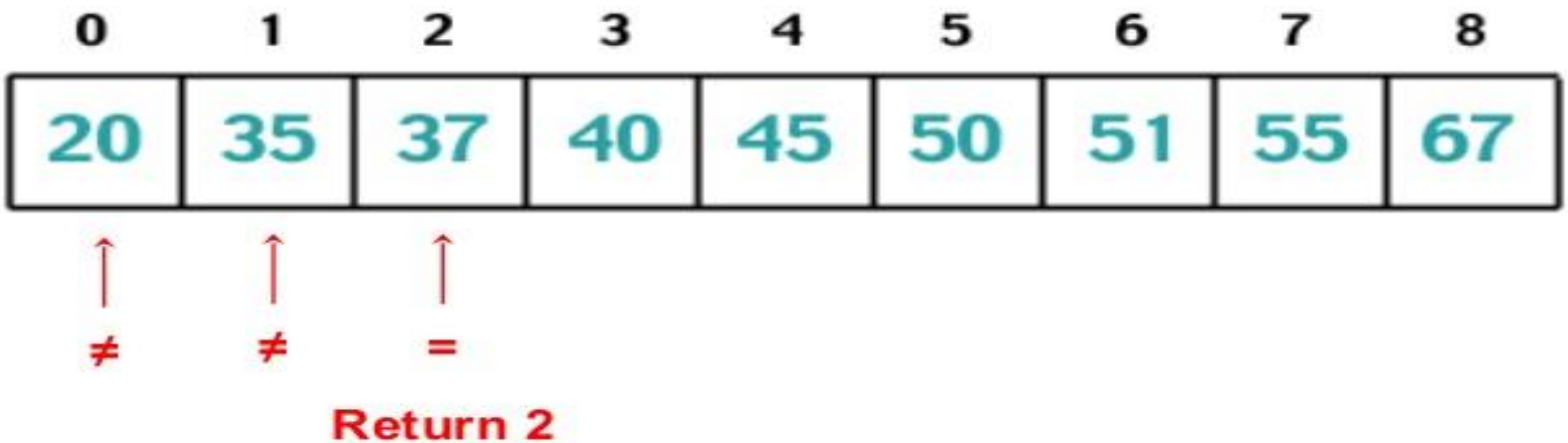
**Algorithm: Linear-Search (L, N, Key)**

1. Set k := 1 and Loc := 0
2. Repeat Steps 3 and 4 while k <= N
3. If Key = L[k], then Set Loc := k, print: Loc and exit.
4. Set k := k+1
5. If Loc = 0 then Write: Item is not in List
6. Exit

> Here
> L - The list of data items
> N – Total no. of items in L
> Key – Item to be searched.

# Linear Search

- Find 37?



# Example of Linear Search

List : 10, 6, 222, 44, 55, 77, 24

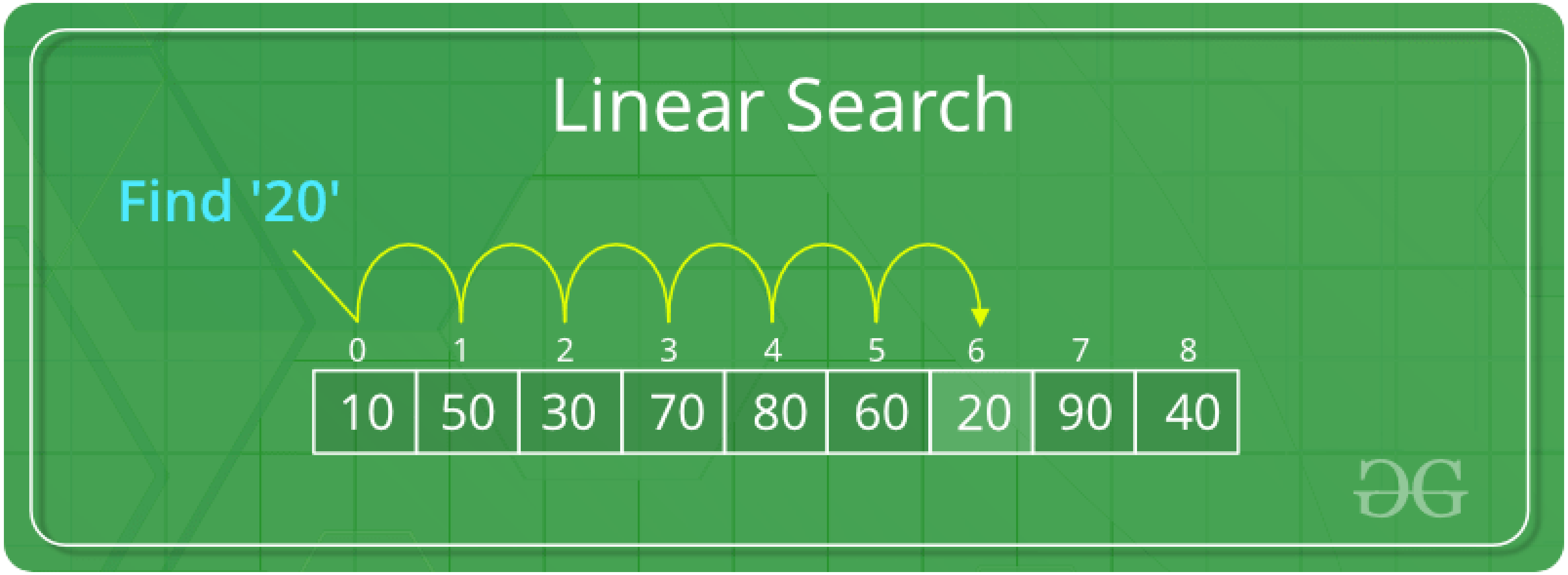
Key: 55

**Steps:**

| | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| 1. | 55 | $\boxed{10}$ | 6 | 222 | 44 | 55 | 77 | 24 | → | K=1 |
| 2. | 55 | 10 | $\boxed{6}$ | 222 | 44 | 55 | 77 | 24 | → | K=2 |
| 3. | 55 | 10 | 6 | $\boxed{222}$ | 44 | 55 | 77 | 24 | → | K=3 |
| 4. | 55 | 10 | 6 | 222 | $\boxed{44}$ | 55 | 77 | 24 | → | K=4 |
| 5. | 55 | 10 | 6 | 222 | 44 | $\boxed{55}$ | 77 | 24 | → | K=5 |

**So, Item 55 is in position 5.**

https://www.google.com/imgres?imgurl=https%3A%2F%2Fwww.tutorialspoint.com%2Fdata_structures_algorithms%2Fimages%2Flinear_search.gif&imgrefurl=https%3A%2F%2Fwww.tutorialspoint.com%2Fdata_structures_algorithms%2Flinear_search_algorithm.htm&tbnid=8quzOEPl2h3ZaM&vet=12ahUKEwjCq-a1_JDqAhXg5zgGHb6vBsMQMygBegUIARCnAQ..i&docid=A-OYa3qiq8RL7M&w=438&h=180&q=example%20linear%20search%20data%20structures&ved=2ahUKEwjCq-a1_JDqAhXg5zgGHb6vBsMQMygBegUIARCnAQ



# Binary Search

- List of items should be sorted.

- It uses the divide-and-conquer technique.

- The elements of the list are not necessarily all unique. If one searches for a value that occurs multiple times in the list, the index returned by the binary search will be of the first-encountered equal element.

- Compare the given Key value with the element in the middle of the list. It tells which half of the list contains the Item.Then compare Key with the element in the middle of the correct half to determine which quarter of the list contains Item. Continue the process until finding Item in the list.

## **<u>Algorithm:</u>** **Binary-Search (L, N, Key)**

1. Set Loc := 0, Beg := 1, End :=N and Mid := (Beg + End)/2
2. Repeat steps 3 to 6 while Beg <= End
3. If Key < L[Mid], then Set End := Mid – 1
4. Else if Key>L[Mid] then Set Beg := Mid + 1
5. Else if Key = L[Mid] then Loc:= Mid, print: Loc and exit.
6. Set Mid := (Beg + End)/2
7. If Loc = 0 Write: Item is not in List.
8. Exit.

Here
L - The list of data items
N – Total no. of items in L
Key – Item to be searched.

## Example of Binary Search

List : 6, 10, 20, 30, 55, 75, 190

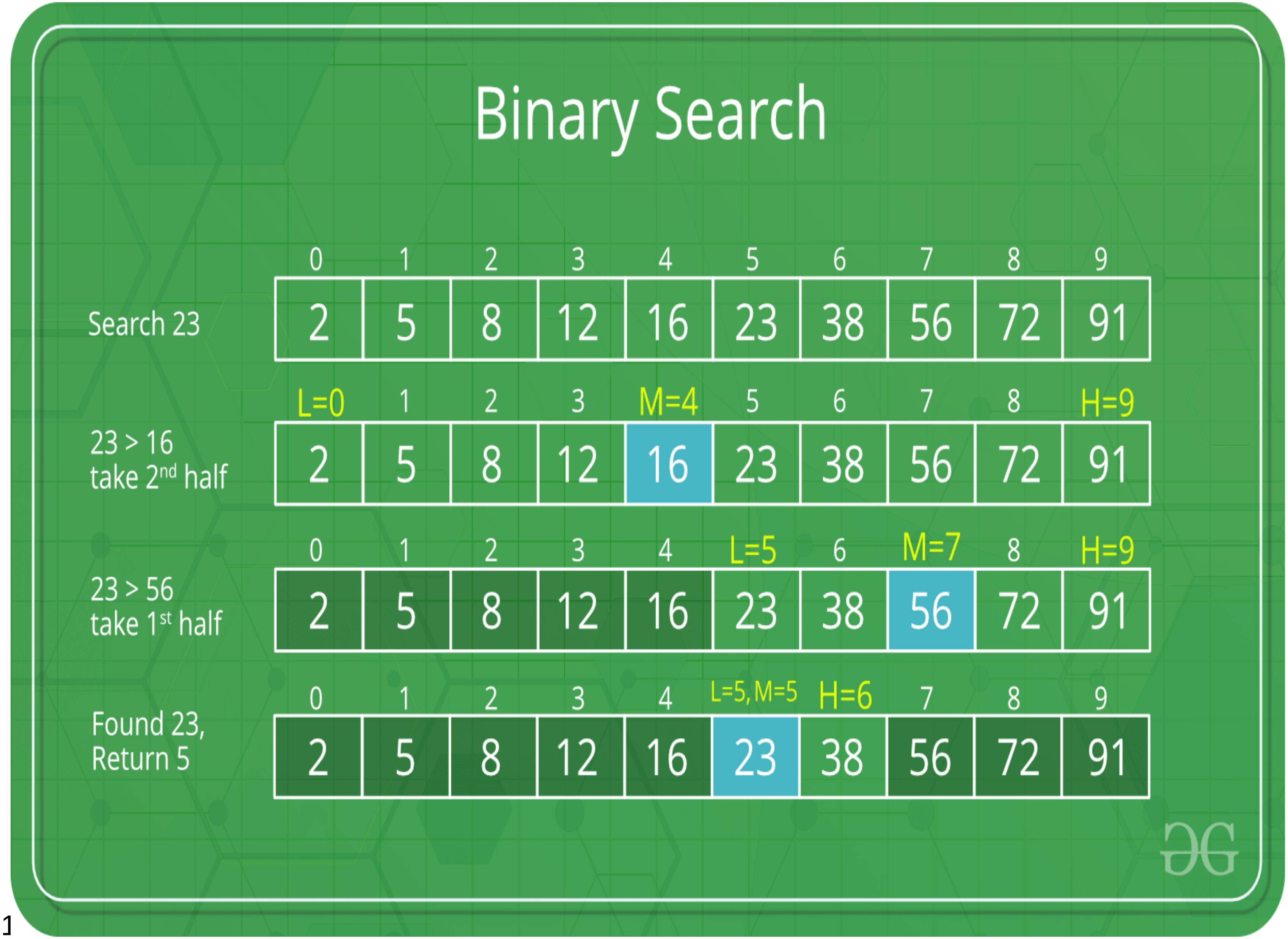
Key: 55

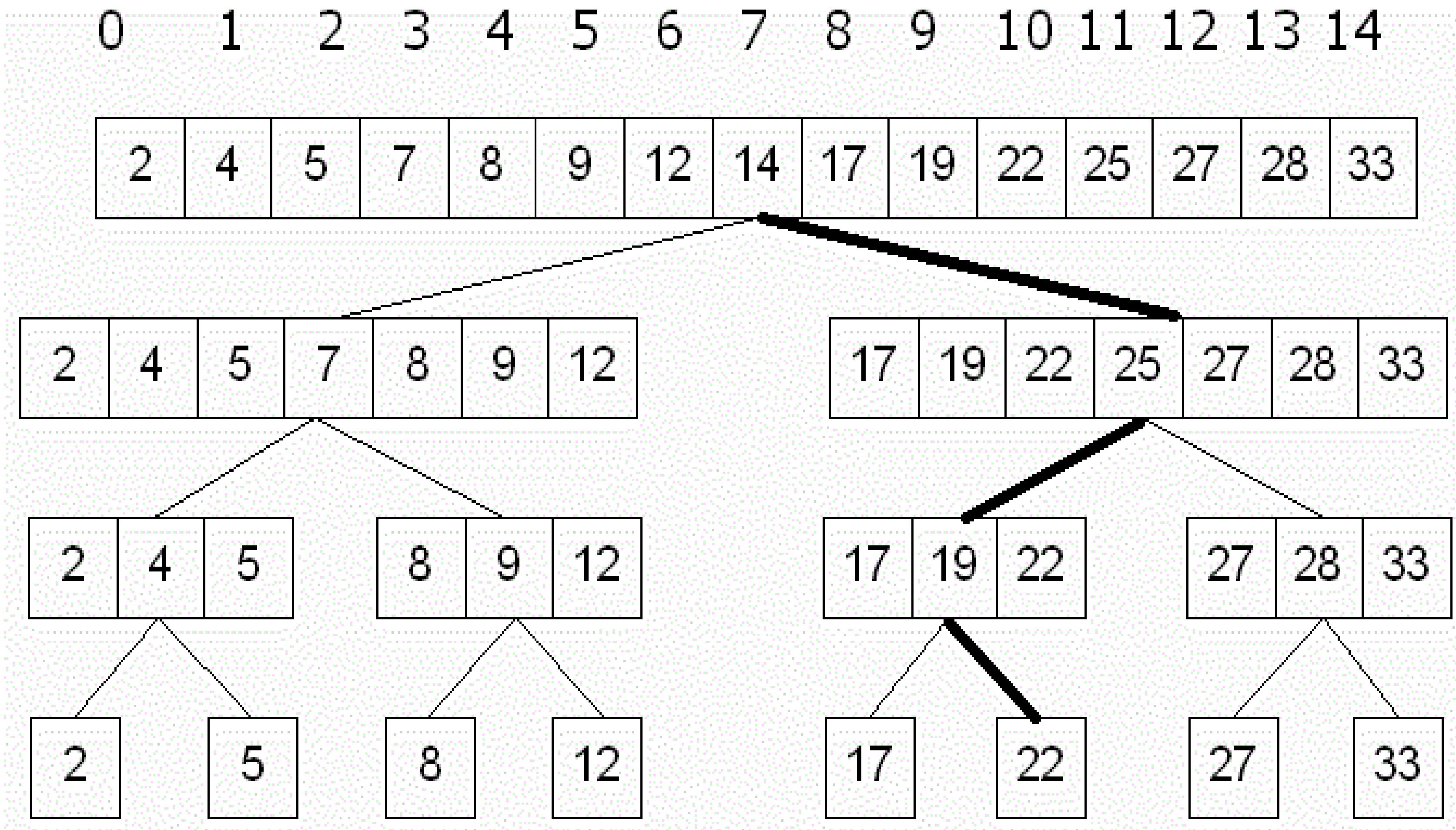
Steps:

| | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| 1. | 55 | 6 | 10 | 20 | $\boxed{30}$ | 55 | 75 | 190 | Beg=1, End=7,Mid=4 |
| 2. | 55 | 6 | 10 | 20 | 30 | 55 | $\boxed{75}$ | 190 | Beg=5, End=7,Mid=6 |
| 3. | 55 | 6 | 10 | 20 | 30 | $\boxed{55}$ | 75 | 190 | Beg=5, End=5,Mid=5 |

[Item Found]

**So, Item 55 is in position 5.**


Binary Search
0 1 2 3 4 5 6 7 8 9
Search 23
2 5 8 12 16 23 38 56 72 91
L=0 1 2 3 M=4 5 6 7 8 H=9
23 > 16
take 2nd half
2 5 8 12 16 23 38 56 72 91
0 1 2 3 4 L=5 6 M=7 8 H=9
23 > 56
take 1st half
2 5 8 12 16 23 38 56 72 91
0 1 2 3 4 L=5, M=5 H=6 7 8 9
Found 23,
Return 5
2 5 8 12 16 23 38 56 72 91


0 1 2 3 4 5 6 7 8 9 10 11 12 13 14
2 4 5 7 8 9 12 14 17 19 22 25 27 28 33
2 4 5 7 8 9 12
17 19 22 25 27 28 33
2 4 5
8 9 12
17 19 22
27 28 33
2
5
8
12
17
22
27
33

# Binary Search

**Example 4.9**

Let DATA be the following sorted 13-element array:

DATA: 11, 22, 30, 33, 40, 44, 55, 60, 66, 77, 80, 88, 99

We apply the binary search to DATA for different values of ITEM.

**(a)** Suppose ITEM = 40. The search for ITEM in the array DATA is pictured in Fig. 4.6, where the values of DATA[BEG] and DATA[END] in each stage of the

# Binary Search

algorithm are indicated by circles and the value of DATA[MID] by a square. Specifically, BEG, END and MID will have the following successive values:

1. Initially, BEG = 1 and END = 13. Hence

   MID = INT[(1 + 13)/2] = 7 and so DATA[MID] = 55

2. Since 40 < 55, END has its value changed by END = MID − 1 = 6. Hence

   MID = INT[(1 + 6)/2] = 3 and so DATA[MID] = 30

3. Since 40 > 30, BEG has its value changed by BEG = MID + 1 = 4. Hence

   MID = INT[( 4 + 6)/2] = 5 and so DATA[MID] = 40

   We have found ITEM in location LOC = MID = 5.

(1) 11, 22, 30, 33, 40, 44, $\boxed{55,}$ 60, 66, 77, 80, 88, 99

(2) 11, 22, $\boxed{30,}$ 33, 40, 44, 55, 60, 66, 77, 80, 88, 99

(3) 11, 22, 30, 33, $\boxed{40,}$ 44, 55, 60, 66, 77, 80, 88, 99 [Successful]

**Fig. 4.6** *Binary Search for ITEM = 40*

# Binary Search

**(b)** Suppose ITEM = 85. The binary search for ITEM is pictured in Fig. 4.7. Here BEG, END and MID will have the following successive values:

1. Again initially, BEG = 1, END = 13, MID = 7 and DATA[MID] = 55.
2. Since 85 > 55, BEG has its value changed by BEG = MID + 1 = 8. Hence

   MID = INT[(8 + 13)/2] = 10 and so DATA[MID] = 77

3. Since 85 > 77, BEG has its value changed by BEG = MID + 1 = 11. Hence

   MID = INT[(11 + 13)/2] = 12 and so DATA[MID] = 88

4. Since 85 < 88, END has its value changed by END = MID − 1 = 11. Hence

   MID = INT[(11 + 11)/2] = 11 and so DATA[MID] = 80

   (Observe that now BEG = END = MID = 11.)

Since 85 > 80, BEG has its value changed by BEG = MID + 1 = 12. But now BEG > END. Hence ITEM does not belong to DATA.

# Binary Search

(1) 11, 22, 30, 33, 40, 44, $\boxed{55,}$ 60, 66, 77, 80, 88, 99

(2) 11, 22, 30, 33, 40, 44, 55, 60, 66, $\boxed{77,}$ 80, 88, 99

(3) 11, 22, 30, 33, 40, 44, 55, 60, 66, 77, 80, $\boxed{88,}$ 99

(4) 11, 22, 30, 33, 40, 44, 55, 60. 66, 77, 80, 88, 99 [Unsuccessful]

**Fig. 4.7** *Binary Search for ITEM = 85*

**Item to be searched = 23**

Step 1

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| **1** | **5** | **7** | **8** | **13** | **19** | **20** | **23** | **29** |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |

**a [mid] = 13**
**13 < 23**
**beg = mid + 1 = 5**
**end = 8**
**mid = (beg + end)/2 = 13 / 2 = 6**

Step 2

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| **1** | **5** | **7** | **8** | **13** | **19** | **20** | **23** | **29** |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |

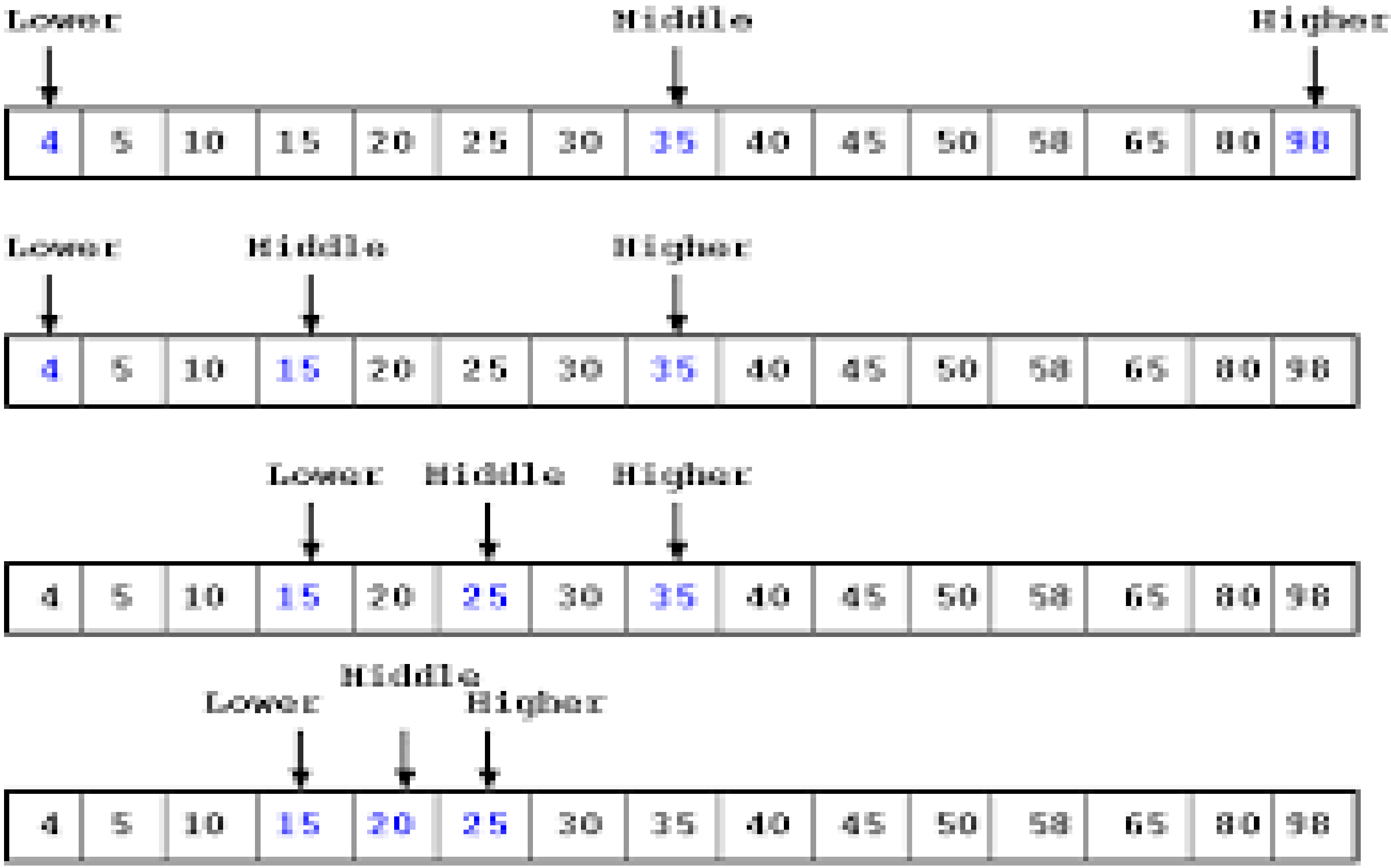
**a [mid] = 20**
**20 < 23**
**beg = mid + 1 = 7**
**end = 8**
**mid = (beg + end)/2 = 15 / 2 = 7**

Step 3

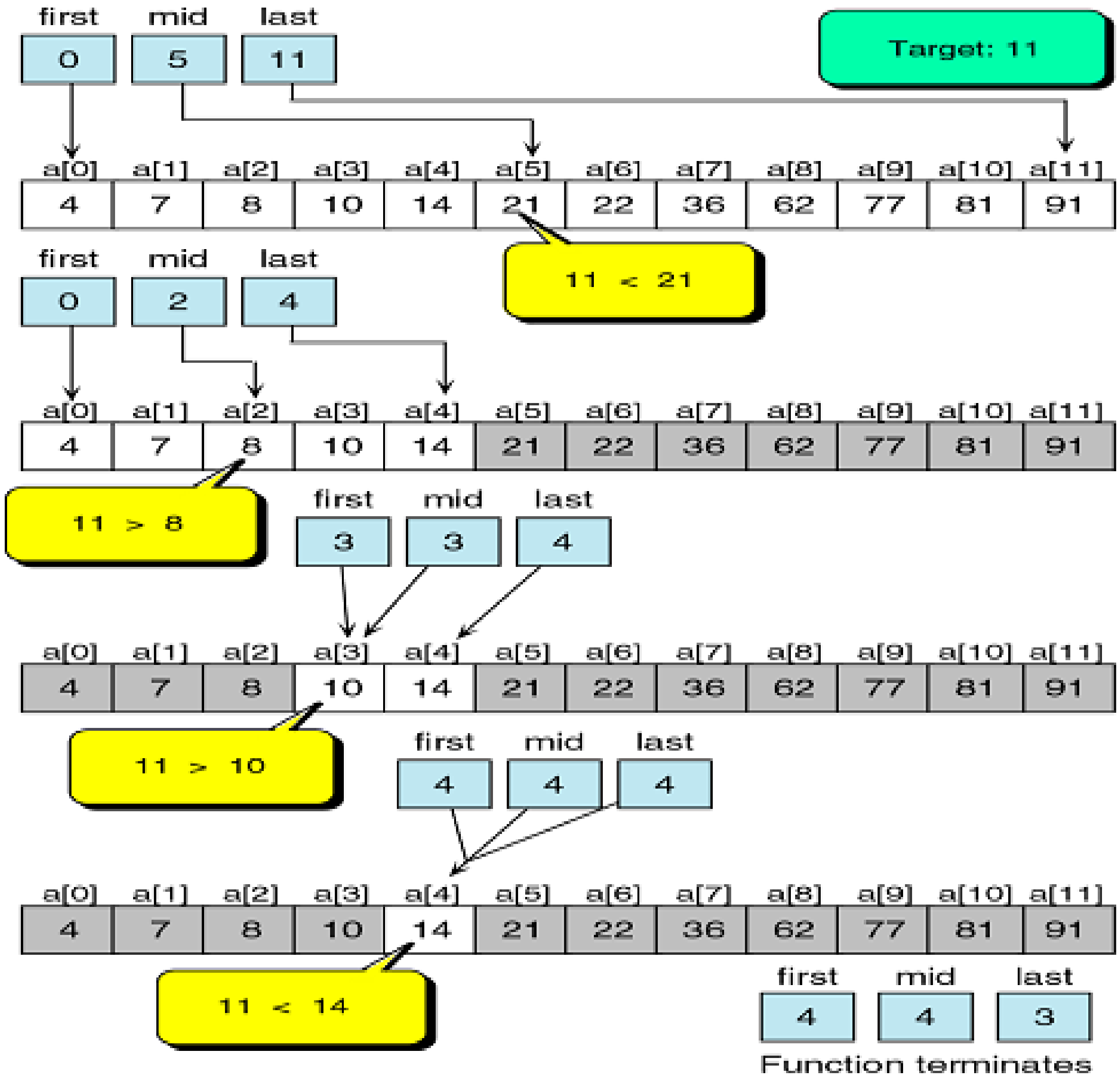
| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| **1** | **5** | **7** | **8** | **13** | **19** | **20** | **23** | **29** |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |

**a [mid] = 23**
**23 = 23**
**loc = mid**

**Return location 7**



Lower
Middle
Higher
4 5 10 15 20 25 30 35 40 45 50 58 65 80 98
Lower
Middle
Higher
4 5 10 15 20 25 30 35 40 45 50 58 65 80 98
Lower
Middle
Higher
4 5 10 15 20 25 30 35 40 45 50 58 65 80 98
Middle
Lower
Higher
4 5 10 15 20 25 30 35 40 45 50 58 65 80 98



first mid last
0 5 11
Target: 11
a[0] a[1] a[2] a[3] a[4] a[5] a[6] a[7] a[8] a[9] a[10] a[11]
4 7 8 10 14 21 22 36 62 77 81 91
11 < 21
first mid last
0 2 4
a[0] a[1] a[2] a[3] a[4] a[5] a[6] a[7] a[8] a[9] a[10] a[11]
4 7 8 10 14 21 22 36 62 77 81 91
11 > 8
first mid last
3 3 4
a[0] a[1] a[2] a[3] a[4] a[5] a[6] a[7] a[8] a[9] a[10] a[11]
4 7 8 10 14 21 22 36 62 77 81 91
11 > 10
first mid last
4 4 4
a[0] a[1] a[2] a[3] a[4] a[5] a[6] a[7] a[8] a[9] a[10] a[11]
4 7 8 10 14 21 22 36 62 77 81 91
11 < 14
first mid last
4 4 3
Function terminates

**<u>END</u>**